



Market Data Portal (MDP) — Technical Documentation Index

Coresight Research Portal — Technical Documentation

Document version: 1.0

Generated: June 29, 2026

Classification: Internal — Engineering Reference

CONFIDENTIAL — INTERNAL USE ONLY — DO NOT DISTRIBUTE

This document contains proprietary information belonging to Coresight Research. Unauthorized disclosure, copying, or distribution is strictly prohibited.

Table of Contents

Market Data Portal (MDP) — Technical Documentation Index	1
How to Read These Docs	3
Architecture Overview	3
Shared Infrastructure Documentation	6
Page Registry (main.py)	6
Authentication Summary	8
Database Summary	9
Startup Warmups (authenticated only)	9
Key External Integrations	10
Documentation Folder Map	11
Application Directory Map	11
PDF Versions	11
Documentation Conventions	14

Application: Market Data Portal (MDP)

Stack: Streamlit multipage app · Python 3 · SQLAlchemy 2 · Azure MySQL Flexible Server · OpenID Connect (OIDC) via Coresight Identity Provider (IdP) · Azure Blob Storage · MCP servers

How to Read These Docs

This folder documents the **Market Data Portal (MDP)** for engineering review. Use it in this order:

1. **Start here (README)** — architecture, auth summary, page registry, warmup flags.
2. **Infrastructure (`00-infrastructure/`)** — shared bootstrap, components, core, data layer, and utils. Read before diving into any page doc.
3. **Page docs (`pages/`)** — one markdown file per registered route; each opens with architecture diagrams, then tables for dependencies, session state, and data layer.
4. **Diagrams first** — every doc leads with Mermaid flowcharts (`flowchart TD/LR`) for sequences and pipelines, and `sequenceDiagram` for multi-actor auth and Application Programming Interface (API) flows. Color legend is consistent across all docs (see Architecture Overview).
5. **PDFs** — generated exports live under `_pdf/` (individual per-doc PDFs in `_pdf/individual/` , merged bundle at `_pdf/Market-Data-Portal-Technical-Documentation-Complete.pdf`). Diagrams are **rendered as images** (not code blocks). Regenerate with `scripts/generate_technical_doc_pdf.py` (see [Generating PDFs](#)).

Conventions: paths are repo-relative (`app/pages/...`). Table names use the `coreiq_` prefix. Auth cookie `auth_session` is the runtime source of truth; `market_data_user_sessions` is audit-only.

Architecture Overview

The portal is a Streamlit multipage application. Every browser request executes `app/main.py` first, which bootstraps authentication, database connections, and background warmups before delegating to the active page module.

High-level map from browser request through `main.py` , application layers, and external integrations.

Figure 1 Part A — Architecture layers

Figure 1 Part A — Architecture layers

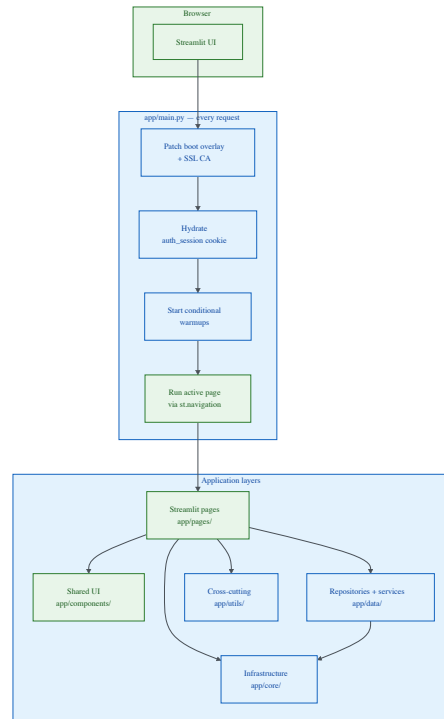
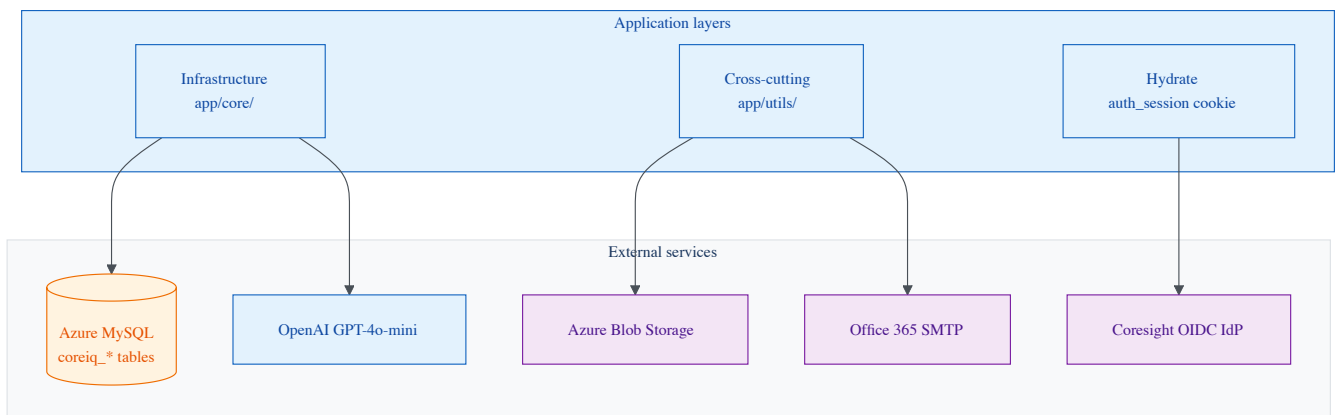


Figure 1 Part B — Data flow to external services

Figure 1 Part B — Data flow to external services



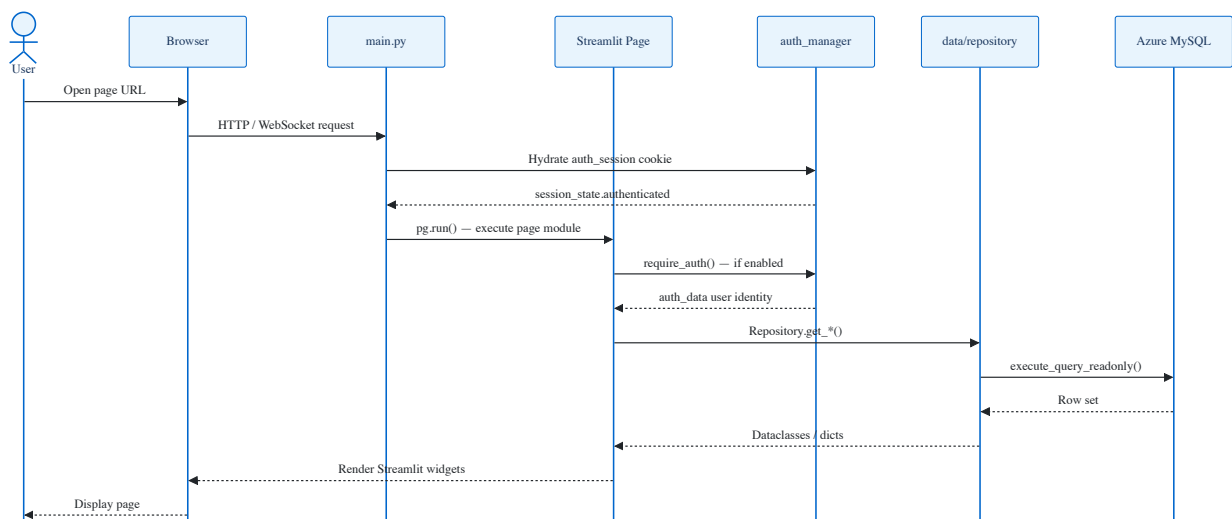
Layer responsibilities

Layer	Path	Responsibility
Entry	<code>app/main.py</code>	Page registration, auth bootstrap, startup warmups
Pages	<code>app/pages/</code>	One file = one Streamlit route; page-specific User Interface (UI) and orchestration
Components	<code>app/components/</code>	Shared UI: navigation, styles, toolbar, company profile
Data	<code>app/data/</code>	Models, repositories, screening engine, forecasting services
Core	<code>app/core/</code>	Config, database, auth, Access Control List (ACL), Large Language Model (LLM) extraction, search
Utils	<code>app/utils/</code>	Logging, caching, Azure Blob, Portable Document Format (PDF), email, constants

Data flow (typical authenticated page)

Typical authenticated request: cookie hydrate in `main.py`, optional `require_auth()` on the page, then a repository read via the AUTOCOMMIT read engine.

Typical authenticated request: cookie hydrate in `main.py`, optional `requireauth()` on the page, then a repository read via the AUTOCOMMIT read engine.



Shared Infrastructure Documentation

Document	Scope
00-app-bootstrap-and-auth.md	<code>main.py</code> startup, page registration, OIDC auth flow
01-shared-components.md	<code>app/components/</code> — navigation, styles, toolbar, charts, tables
02-core-infrastructure.md	<code>app/core/</code> — config, database, auth, ACL, LLM, search
03-data-layer.md	<code>app/data/</code> — models, repository, screening, forecasting
04-utils-and-caching.md	<code>app/utils/</code> — logger, cache, Azure Blob, PDF, email

Page Registry (`main.py`)

Pages are registered in `_PAGES_CONFIG` and exposed via `st.navigation(..., position="hidden")`. The default landing page is **Login**.

Page file	Title	URL path	Detailed doc
<code>pages/login.py</code>	Login	<code>/login</code>	login.md
<code>pages/logout_bridge.py</code>	Logout	<code>/logout_bridge</code>	logout-bridge.md
<code>pages/home.py</code>	Home	<code>/home</code>	home.md
<code>pages/market_data.py</code>	Market Data	<code>/market_data</code>	market-data.md
<code>pages/newsroom.py</code>	Newsroom	<code>/newsroom</code>	newsroom.md
<code>pages/earnings_calls.py</code>	Earnings Calls	<code>/earnings_calls</code>	earnings-calls.md
<code>pages/live_earnings_transcript.py</code>	Live Transcript	<code>/live_earnings_transcript</code>	live-earnings-transcript.md
<code>pages/earnings_calendar.py</code>	Earnings Calendar	<code>/earnings_calendar</code>	earnings-calendar.md
<code>pages/screening.py</code>	Screening	<code>/screening</code>	screening.md
<code>pages/company_filings.py</code>	Company Filings	<code>/company_filings</code>	company-filings.md
<code>pages/company_filings_add_files.py</code>	Add Files	<code>/company_filings_add_files</code>	company-filings-add-files.md
<code>pages/logs.py</code>	Logs	<code>/logs</code>	logs.md
<code>pages/retailer_adding.py</code>	Retailers	<code>/retailer_adding</code>	retailer-adding.md
<code>pages/forecasting.py</code>	Forecasting	<code>/forecasting</code>	forecasting.md
<code>pages/access_management.py</code>	Access Management	<code>/access_management</code>	access-management.md

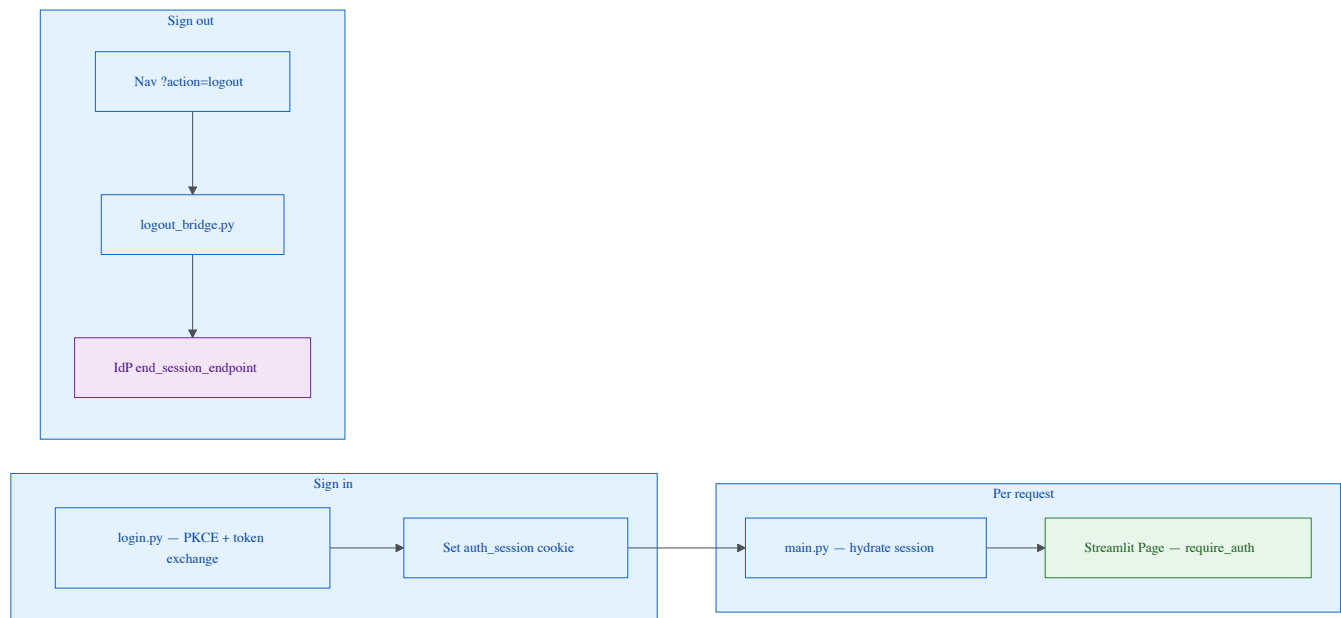
Unregistered pages (exist in `app/pages/` but not in `main.py`)

Page file	Notes	Detailed doc
<code>admin_iam.py</code>	Admin IAM — ACL-gated	—
<code>audit.py</code>	Audit log viewer — admin	—
<code>forecasting_admin.py</code>	Forecast admin — ACL-gated	forecasting-admin.md
<code>reports.py</code>	Reports — admin	—
<code>settings.py</code>	Settings — admin	—

Authentication Summary

OpenID Connect (OIDC)-only via Coresight Identity Provider (IdP). Cookie `auth_session` is the source of truth; database `market_data_user_sessions` is audit-only.

OpenID Connect (OIDC)-only via Coresight Identity Provider (IdP). Cookie `authsession` is the source of truth; database `marketdatausersessions` is audit-only.



Authentication is enforced via `main.py` cookie hydration (`is_authenticated()` on every request) and page-level `require_auth()` where enabled. Pages that do not call `require_auth()` still receive identity from the central bootstrap but do not redirect unauthenticated visitors to `/login`. See [00-app-bootstrap-and-auth.md](#) for the full page matrix.

See [00-app-bootstrap-and-auth.md](#) and [02-core-infrastructure.md](#) for full detail.

Database Summary

Two SQLAlchemy engines in `core/database.py` :

Engine	Use	RTT on Azure
Read (<code>_read_engine</code> , AUTOCOMMIT)	All SELECTs, single-statement writes	~1 RTT
Main (<code>_engine</code> , transactional)	<code>get_session()</code> , multi-query batches	~3 RTTs

All tables prefixed `coreiq_` (55 tables). See [02-core-infrastructure.md](#) and [03-data-layer.md](#).

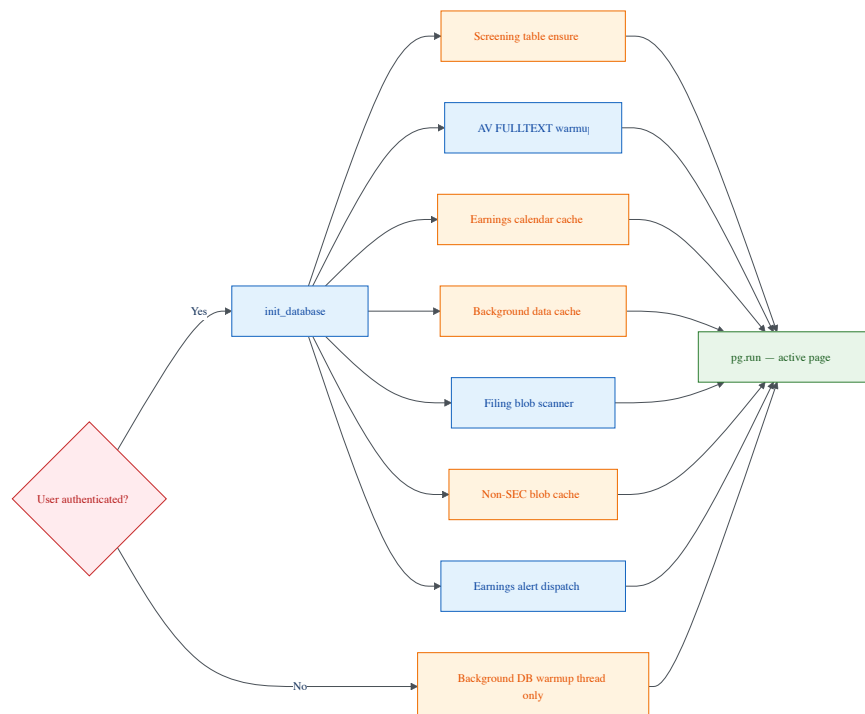
Startup Warmups (authenticated only)

Warmup	Module	Env gate
Database init	<code>core.database.init_database</code>	Always when authenticated
Screening tables	<code>watchlist/saved_criteria/portal/earnings_alert</code> <code>ensure_tables</code>	<code>APP_SCREENING_PREWARM_STARTED</code>
Alpha Vantage (AV) FULLTEXT	<code>warmup_av_fulltext</code>	<code>ENABLE_AV_FT_WARMUP=1</code>
Earnings calendar cache	<code>repository.warmup_ec_caches</code>	<code>ENABLE_EC_WARMUP=1</code>
Background data cache	<code>cache_manager.start_background_warmup</code>	<code>ENABLE_BG_WARMUP=1</code>
Filing blob scanner	<code>background_scanner</code>	<code>ENABLE_BG_SCANNER=1</code>
Non-U.S. Securities and Exchange Commission (SEC) filing cache	<code>non_sec_blob_fallback</code>	Once per process
Earnings alert dispatch	<code>earnings_alert_dispatcher</code>	<code>EARNINGS_ALERT_DISPATCH=1</code>

Unauthenticated requests (login page) only get a lightweight background Database (DB) warmup thread.

Authenticated warmups run in parallel background threads after `init_database()` :

Authenticated warmups run in parallel background threads after `initdatabase()`:



Key External Integrations

Integration	Env vars	Purpose
Coresight OIDC	IDP_BASE_URL , OIDC_CLIENT_ID , OIDC_CLIENT_SECRET , OIDC_REDIRECT_URI	Authentication
Azure MySQL	STG_DB_* or PROD_DB_*	All persistent data
Azure Blob	AZURE_STORAGE_ACCOUNT_NAME , AZURE_STORAGE_ACCOUNT_KEY , AZURE_BLOB_CONTAINER	Filing PDFs
OpenAI	OPENAI_API_KEY	eXtensible Business Reporting Language (XBRL) metric LLM extraction
Simple Mail Transfer Protocol (SMTP)	SMTP_SERVER , EMAIL_PASSWORD , FROM_EMAIL	Earnings alert emails

Documentation Folder Map

```
docs/technical-documentation/
├─ README.md                # This index
├─ FOLDER-STRUCTURE.md      # Reusable taxonomy template for future products
├─ 00-infrastructure/       # Mirrors app/core, app/data, app/utils, app/components, main.py
│   └─ 00-app-bootstrap-and-auth.md
│   └─ 01-shared-components.md
│   └─ 02-core-infrastructure.md
│   └─ 03-data-layer.md
│   └─ 04-utils-and-caching.md
├─ pages/                   # One doc per Streamlit route (mirrors app/pages/)
│   └─ login.md
│   └─ home.md
│   └─ ...
├─ _pdf/                    # Generated PDF exports (do not edit by hand)
│   └─ individual/          # Per-document PDFs
│       └─ Market-Data-Portal-Technical-Documentation-Complete.pdf
├─ _templates/              # PDF HTML/CSS templates for Playwright pipeline
├─ _assets/                 # Logo and static assets for PDF cover pages
├─ _diagram_cache/          # Cached Mermaid SVG renders
└─ _mermaid_config.json     # Mermaid CLI theme config
```

Application Directory Map

```
app/
├─ main.py                  # Entry point
├─ pages/                   # One file per route
├─ components/              # Shared UI
├─ core/                    # Infrastructure
├─ data/                    # Models + repositories + services
├─ utils/                   # Cross-cutting utilities
└─ assets/                  # Static SVG/PNG icons
```

PDF Versions

PDF exports mirror markdown sources under `_pdf/individual/`. PDFs include a branded cover page, table of contents, rendered Mermaid diagrams, headers/footers with confidentiality notice, and Coresight styling.

Complete merged PDF

A single production bundle combines all 22 documents with a master cover, clickable table of contents, and section dividers:

Market-Data-Portal-Technical-Documentation-Complete.pdf

Individual per-document PDFs are in `_pdf/individual/` for targeted reference. Regenerate the merged bundle with:

```
python scripts/merge_technical_doc_pdf.py
```

Generating PDFs

Prerequisites (one-time):

```
pip install markdown          # in project venv
npx @mermaid-js/mermaid-cli   # auto-installed on first run via npx
playwright install chromium   # if not already installed for UI tests
```

Regenerate all 22 PDFs:

```
python scripts/generate_technical_doc_pdf.py
```

Regenerate one document:

```
python scripts/generate_technical_doc_pdf.py screening.md
```

Verify diagrams embedded (not raw code):

```
python scripts/generate_technical_doc_pdf.py --verify screening.pdf
```

Validate all PDFs (blank pages, diagram splits, clickable TOC samples):

```
python scripts/validate_technical_doc_pdfs.py
```

Pipeline: Markdown → Mermaid pre-render (`mmdc`) → SVG → branded HyperText Markup Language (HTML) (`_templates/_pdf_template.html` + `_templates/_pdf_styles.css`) → Playwright PDF with cover, clickable table of contents (H1/H2 with dotted leaders and page numbers), headers, and footers → PyMuPDF internal link annotations.

Assets: Logo at `_assets/coresight-logo.png` ; diagram cache at `_diagram_cache/` ; TOC validation screenshots at `_validation-toc/` .

Infrastructure

Markdown	PDF
README.md	_pdf/individual/README.pdf
00-app-bootstrap-and-auth.md	_pdf/individual/00-app-bootstrap-and-auth.pdf
01-shared-components.md	_pdf/individual/01-shared-components.pdf
02-core-infrastructure.md	_pdf/individual/02-core-infrastructure.pdf
03-data-layer.md	_pdf/individual/03-data-layer.pdf
04-utils-and-caching.md	_pdf/individual/04-utils-and-caching.pdf

Registered pages

Markdown	PDF
login.md	_pdf/individual/login.pdf
logout-bridge.md	_pdf/individual/logout-bridge.pdf
home.md	_pdf/individual/home.pdf
market-data.md	_pdf/individual/market-data.pdf
newsroom.md	_pdf/individual/newsroom.pdf
earnings-calls.md	_pdf/individual/earnings-calls.pdf
live-earnings-transcript.md	_pdf/individual/live-earnings-transcript.pdf
earnings-calendar.md	_pdf/individual/earnings-calendar.pdf
screening.md	_pdf/individual/screening.pdf
company-filings.md	_pdf/individual/company-filings.pdf
company-filings-add-files.md	_pdf/individual/company-filings-add-files.pdf
logs.md	_pdf/individual/logs.pdf
retailer-adding.md	_pdf/individual/retailer-adding.pdf
forecasting.md	_pdf/individual/forecasting.pdf
access-management.md	_pdf/individual/access-management.pdf

Unregistered pages

Markdown	PDF
forecasting-admin.md	_pdf/individual/forecasting-admin.pdf

Documentation Conventions

Element	Convention
Diagrams	Mermaid <code>flowchart TD</code> for vertical flows, <code>flowchart LR</code> for pipelines, <code>sequenceDiagram</code> for multi-actor auth/API
Colors	<code>start</code> blue · <code>ui</code> green · <code>db</code> orange · <code>ext</code> purple · <code>error</code> red · <code>process</code> grey
Paths	Relative to repository root (<code>app/...</code>)
Tables	<code>coreiq_*</code> prefix on all MySQL tables
Auth	<code>auth_session</code> cookie = runtime truth; DB session table = audit only